Инструменты и средства программирования

Лабораторная работа №6

Сборки и метаданные

Цель работы: знакомство с метаданными сборок .Net

Задача работы: научиться с помощью Reflection динамически находить нужные типы в загруженных сборках.

Время выполнения работы: 2 часа

Результат выполнения работы: программа, обеспечивающая функционал согласно заданию.

1. Выполнение работы

1.1. Создание консольного приложения

Создайте консольное приложение. Имя решения «XXX», имя проекта «XXX.Client», где XXX – ваша фамилия

1.2. Создание библиотеки классов предметной области

Добавьте в решение проект – библиотеку классов .Net с именем XXX.Domain.

В созданной библиотеке опишите класс предметной области согласно варианту задания, содержащий любые свойства типа ***int*** и ***bool*** и свойство ***Name*** (имя) типа ***string***

Созданный класс должен реализовать интерфейс IEquatable<T>

1.3. Создание библиотеки Mediator

Добавьте в решение проект – библиотеку классов .Net с именем XXX.Mediator.

В созданной библиотеке необходимо реализовать функционал шаблона медиатор. Для этого:

a. Опишите интерфейсы запросов (сообщений):

```
/// <summary>
/// Запрос, не возвращающий данные
/// </summary>
public interface IRequest {}

/// <summary>
/// Запрос, возвращающий данные
/// </summary>
/// <typeparam name="TResponse">Тип возвращаемых данных</typeparam>
public interface IRequest<out TResponse> {}
```

b. Опишите интерфейсы обработчиков запросов (сообщений):

```
/// <summary>
/// Обработчик запроса
/// </summary>
/// <typeparam name="TRequest">Тип запроса</typeparam>
public interface IRequestHandler<in TRequest>
        where TRequest : IRequest
{
    void Handle(TRequest request);
}
/// <summary>
/// Обработчик запроса, возвращающий данные
/// </summary>
/// <typeparam name="TRequest">Тип запроса</typeparam>
/// <typeparam name="TResponse">Тип возвращаемых данных</typeparam>
public interface IRequestHandler<in TRequest, TResponse>
        where TRequest : IRequest<TResponse>
{
    TResponse Handle(TRequest request);
}
```

c. Опишите интерфейс медиатора (отправителя сообщений):

```
public interface ISender
{
    /// <summary>
    /// Обработка запроса
    /// </summary>
    /// <typeparam name="TRequest">Тип запрос</typeparam>
    /// <param name="request">Объект запроса</param>
    /// <param name="cancellationToken"></param>
```

```
    void Send<TRequest>(TRequest request)
            where TRequest : IRequest;

    /// <summary>
    /// Обработка запроса, возвращающего данные
    /// </summary>
    /// <typeparam name="TResponse">Тип возвращаемых
данных</typeparam>
    /// <param name="request">Объект запроса</param>
    /// <param name="cancellationToken"></param>
    /// <returns> значение типа TResponse</returns>
    TResponse Send<TResponse>(IRequest<TResponse>
request);
}
```

d. Опишите реализацию интерфейса ISender:

```
public class Sender : ISender
{
    /// сборка для поиска хэндлеров
    private readonly Assembly? _assembly;

    /// словарь соответствия типа интерфейса хендлера
    /// и типа (класса) реализации интерфейса хэндлера
    private readonly Dictionary<Type, Type> _handlerImplementations =
new();

    public Sender(Assembly? assembly = null)
    {
        _assembly = assembly ?? Assembly.GetCallingAssembly();
        RegisterHandlers(_assembly);
    }

    /// <summary>
    /// Запись в словарь всех типов интерфейсов хендлеров
    /// и их реализаций, имеющихся в сборке
    /// </summary>
    /// <param name="assembly">сборка для поиска хэндлеров</param>
    private void RegisterHandlers(Assembly assembly)
    {
        . . .
    }

    public void Send<TRequest>(TRequest request)
```

```
                                where TRequest : IRequest
    {
        . . .
    }

    public TResponse Send<TResponse>(IRequest<TResponse> request)
    {
        Type handlerType = typeof(IRequestHandler<,>)
               .MakeGenericType(request.GetType(), typeof(TResponse));
        try
        {
            var handler = CreateHandler(handlerType);
            return handler.Handle((dynamic)request);
        }
        catch (Exception e)
        {
            System.Console.WriteLine(e.Message);
            throw;
        }
    }

    /// <summary>
    /// Получить объект handler
    /// </summary>
    /// <param name="handlerType">Искомый тип</param>
    /// <returns>объект handler</returns>
    /// <exception cref="NotImplementedException"></exception>
    dynamic CreateHandler(Type handlerType)
    {
        var implementationType = handlerImplementations[handlerType];
        if (implementationType is not null)
        {
            return
Activator.CreateInstance(implementationType)!;
        }
        throw new NotImplementedException("Handler not found");
    }
}
```

Пояснения к заданию:

Есть два типа сообщений: не требующие возврата данных (void) и сообщения, возвращающие данные. Соответственно, есть два типа

обработчиков сообщений, и в интерфейсе ISender описаны два метода отправки сообщений.

В конструктор класса **Sender** передается сборка, в которой нужно искать обработчики сообщений. Если сборка не передана, то искать в сборке, которая использует Sender.

В методе **RegisterHandlers** с помощью рефлексии (Reflection) нужно:

- в сборке (переменная _assembly) найти все неабстрактные классы (LINQ запрос Where),

- для найденных классов (использовать SelectMany) получить все интерфейсы, которые класс наследует,

- если класс наследует обобщенный (Generic) интерфейс, и этот интерфейс наследуется от IRequestHandler<> или IRequestHandler<,> , то найденный тип интерфейса и тип класса реализации записать в словарь _handlerImplementations. Для проверки используйте:

```
i.GetGenericTypeDefinition() == typeof(IRequestHandler<>) ||
i.GetGenericTypeDefinition() == typeof(IRequestHandler<,>)
```

Пример использования библиотеки Mediator для вычисления суммы двух чисел

Сообщение:

```
public sealed record CalculateSum(int a, int b):IRequest<int>;
```

Обработчик сообщения:

```
public class CalculateSumHandler : IRequestHandler<CalculateSum, int>
{
    public int Handle(CalculateSum request)
    {
        return request.a + request.b;
    }
}
```

Использование:

```
ISender sender = new Sender();
var result = sender.Send(new CalculateSum(2, 3));
```

```
System.Console.WriteLine($"a + b = {result}");
```

1.4. Создание сборки вариантов использования

Добавьте в решение новый проект – библиотеку классов.

Имя проекта: XXX.UseCases.

Сделайте ссылку на проект XXX.Domain

В созданный проект добавьте файлы SaveData.cs и ReadFile.cs

В файле **SaveData.cs** опишите команду (запрос) и обработчик для записи коллекции объектов (описанных в библиотеке XXX.Domain) в файл:

```
/// <summary>
/// Запрос на сохранение коллекции в файл
/// </summary>
/// <param name="data">Коллекция объектов</param>
/// <param name="fileName">имя файла</param>
public sealed record SaveData(IEnumerable<Book> data, string fileName)
                            : IRequest;

/// <summary>
/// Обработчик запроса
/// </summary>
public class SaveDataHandler : IRequestHandler<SaveData>
{
    public void Handle(SaveData request)
    {
          . . .
    }
}
```

В файле **ReadFile** опишите команду (запрос) и обработчик для чтения коллекции объектов (описанных в библиотеке XXX.Domain) из файла:

```
/// <summary>
/// Запрос на чтение файла
/// </summary>
/// <param name="FileName">Имя файла</param>
public sealed record ReadFile(string FileName)
                                    :IRequest<IEnumerable<Book>>;

/// <summary>
```

```
/// Обработчик запроса чтения файла
/// </summary>
public class ReadFileHandler
                    : IRequestHandler<ReadFile, IEnumerable<Book>>
{
    public IEnumerable<Book> Handle(ReadFile request)
    {
            . . .
    }
}
```

1.4.1. Рекомендации к заданию.

Используйте класс JsonSerializer из пространства имен System.Text.Json

1.5. Использование запросов в клиенте

В классе Program проекта **XXX.Client**:

a) Создайте коллекцию объектов класса, описанного в проекте XXX.Domain

b) Создайте экземпляр ISender. В конструктор класса Sender передайте сборку XXX.UseCases

c) С помощью ISender запишите коллекцию в файл (запрос SaveData)

d) С помощью ISender прочитайте данный из файла (запрос ReadFile)

e) Сравните коллекцию, полученную из файла с исходной коллекцией. Выведите в консоль данные полученной коллекции и сообщение о результате проверки коллекций на равенство содержимого.

1.5.1. Рекомендации к заданию

Ссылку на сборку XXX.UseCases можно получить, например, так:

```
Assembly.GetAssembly(typeof(SaveData))
```

1.6. Варианты предметной области

1. Спортивные команды
2. Исполнители песен
3. Наборы суши

4. Супергерои.
5. Туристические маршруты.
6. Услуги пластического хирурга.
7. Сотрудники предприятия.
8. Товары свадебного салона.
9. Коктейли.
10. Авторы книг.
11. Пациенты больницы.
12. Номинации конкурса.
13. Задачи проекта.
14. Позиции меню пиццерии.
15. Сториз блогера.
16. Автомобили.
17. Экспонаты музея.
18. Процедуры SPA-салона.
19. Виды транспорта транспортной
20. Косметические средства